

COSC 39: Theory of Computation

Problem 1. Register machine.

A k -register machine is a DFA augmented with extra k -bits of memory called registers; the transition of the DFA is determined by both the input bit and the current values in the registers. Formally, a k -register machine M consists of the following components:

- a set of states Q ,
- a starting state $s \in Q$,
- a set of accepting states $A \subseteq Q$,
- an alphabet set Σ ,
- a set of registers $R = \underbrace{\Sigma \times \cdots \times \Sigma}_{k \text{ times}}$, taking values over Σ ,
- a transition function $\delta : Q \times \Sigma \times R \rightarrow Q \times R$, that is, given the current state q , an input character a , and current values of the k registers $r := (r_1, \dots, r_k)$, the transition function outputs the next state $\delta(q, a, r)$ and writes the new values $r' := (r'_1, \dots, r'_k)$ into the registers.

We can define the extended transition function δ^* similarly to the regular DFA. Just like regular DFAs, a k -register machine M accepts a string w if $\delta^*(s, w) \in A$. The language of a k -register machine M is defined to be

$$L(M) := \{w \in \Sigma^* : M \text{ accepts } w\}.$$

Prove that given any language L of some k -register DFA M for constant k , one can construct a regular DFA D that accepts the same set of strings in L . In notation,

$$\forall k\text{-register DFA } M, \exists \text{ DFA } D \text{ such that } L(D) = L(M).$$

Solution.

The key idea for proving such a DFA exists is by conjoining the content of the registers and states within the definition of a regular DFA using tuples.

The immediate issue with proving the statement is that regular DFAs as defined can't store memory directly. However, we can circumvent this problem by encoding the information saved in registers into another part of the DFA: states.

Construct a DFA $D (Q_D, \Sigma, \delta_D, s_D, A_D)$ as follows:

- $Q_D = \text{tuples generated by cross product } Q \times \Sigma^k.$
- $s_D = (s, r_0)$ **where r_0 is the M 's starting register contents.**
- $A_D = \{(q, r) \in Q_D \mid q \in A\}.$
- The transition function δ_D :

$$\delta_D((q, r), a) = (q', r'),$$

where

$$(q', r') = \delta_M(q, a, r).$$

Now, we will prove the condition which our DFA D must satisfy:

$$L(D) = L(M).$$

We will do so by proving the following by induction on the length of w :

Claim: If M reaches state q' with registers r' after reading w , then D reaches state (q', r') after reading w for any string $w \in \Sigma^*$.

Base Case: Let $w = \epsilon$.

By our definition of D , the starting state is (s, r_0) , which is equal to M 's starting state s with starting register contents r_0 . Since both D and M start in the same place, the claim holds.

Inductive Step: Suppose the claim holds for a string w . Let $a \in \Sigma$ and consider wa .

Assume M reaches state q with register values r after reading w . By the inductive hypothesis, D also reaches (q, r) after reading w .

Then on reading a , by definition of transition states:

1. In M , we have $\delta_M(q, a, r) = (q', r')$.
2. In D , we have $\delta_D((q, r), a) = (q', r')$.

So the claim holds for wa and the inductive case is proved. That is for any input string w ,

$$\delta_D^*(s_D, w) = \delta_M^*(s, w)$$

Therefore for any $w \in \Sigma^*$ we have,

$$w \in L(D) \iff \delta_D^*(s_D, w) \in A_D \iff \delta_M^*(s, w) \in A \iff w \in L(M)$$

And we have,

$$L(D) = L(M).$$

Problem 2. Erasing digit sequence.

Let the input be a string of digits from 0 to 9 (in other words, the alphabet set Σ is $\{0, \dots, 9\}$). The ERASE function is defined as follows:

```

ERASE( $w$ ) :
    input: digit string  $w$ 
     $r \leftarrow \epsilon$ 
    while  $w$  is not empty:
         $d \leftarrow$  first digit of  $w$ 
        remove the first digit of  $w$ 
         $r \leftarrow r \cdot d$  // append  $d$  after  $r$ 
        if there are at least  $d$  digits left in  $w$ :
            remove  $d$  digits from  $w$ 
        else:
            return fail
    return  $r$ 

```

A digit string w is *erasable* if $\text{ERASE}(w)$ successfully returns another digit string. For example, the string

$$w = 314159265358979323846264338327950288419$$

is erasable because

$$\text{ERASE}(w) = 355243251.$$

Construct DFAs that recognize the following languages:

- (a) $\{w \in \Sigma^* : w \text{ is erasable}\}$
- (b) $\{w \in \Sigma^* : \text{both } w \text{ and } \text{ERASE}(w) \text{ are erasable}\}$

Note: It is not sufficient to just draw the diagram; you must explain your construction, especially what each state represents, in English (this is equivalent to commenting your code). Alternatively, you may describe the DFAs using the formal tuple $(Q, s, A, \Sigma, \delta)$, but you still need to explain your construction. Answers without English explanations will receive no credit, even when correct.

Solution.

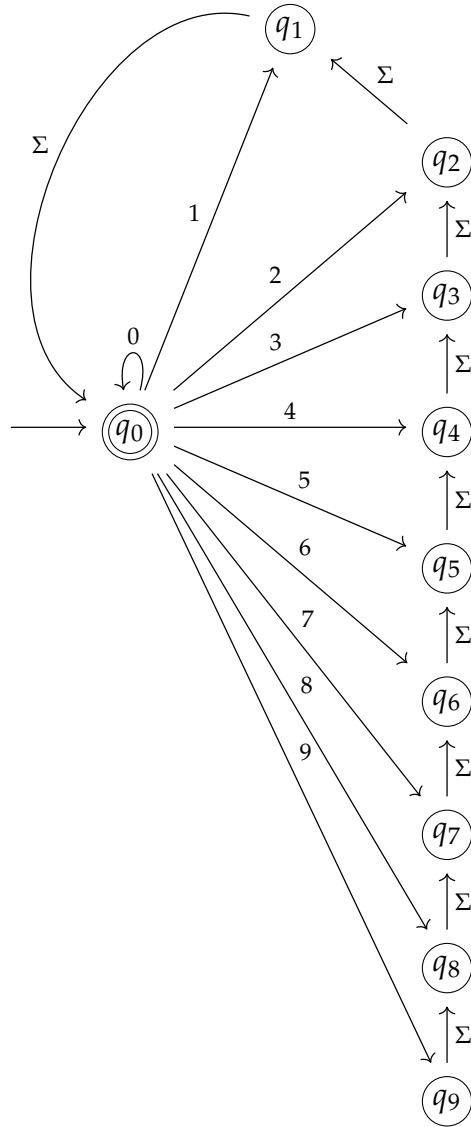
- (a) We construct a DFA with 10 states: a starting (accepting) state q_0 , and states $q_1, \dots, q_9$ forming a counting chain.
 - q_0 : Starting state. It accepts both an empty string, and an 'erasable block'. Reading 0 loops back to this state since we don't need to erase any digits.
 - q_k for $k \in \{1, \dots, 9\}$. These transition states help us skip exactly k characters. If we run out of characters in the input string, then the DFA fails. If we don't, we go back to the starting state having completed an 'erasable block'.

- Transitions:

$$\delta(q_0, d) = \begin{cases} q_0 & \text{if } d = 0 \\ q_d & \text{if } d \in \{1, \dots, 9\} \end{cases}$$

And,

$$\delta(q_k, d) = q_{k-1} \text{ for } k \geq 1, d \in \Sigma$$



(b) To solve this problem, we construct a 1-register machine (from Problem 1 we know that this is equivalent to constructing a DFA). Similar to part (a), the machine has 10 states. Let the current character being read be d .

- q_0 : Starting state. It accepts both an empty string, and a complete sequence of erasable blocks. If $r_1 = 0$, reading digit d begins a erasure block in $\text{ERASE}(w)$'s own erasure, so we update $r_1 \leftarrow d$ (note that we don't decrement in the same loop we assign r_1 a new value). If $r_1 > 0$, the current digit is a header digit erased by $\text{ERASE}(w)$'s erasure, so we decrement $r_1 \leftarrow r_1 - 1$.
- q_k for $k \in \{1, \dots, 9\}$. These transition states help us skip exactly k characters in w , and r_1 remains unchanged. If we run out of characters in the input string, then the DFA fails. If we don't, we go back to the starting state having completed an 'erasable block' for w .
- The terminating condition is when we are at q_0 with no more characters to read and $r_1 = 0$.
- Transitions:

$$\delta(q_0, d, r_1) = \begin{cases} q_0 & \text{if } d = 0 \\ q_d & \text{if } d \in \{1, \dots, 9\} \end{cases}$$

And,

$$\delta(q_k, d, r_1) = q_{k-1} \text{ for } k \geq 1, d \in \Sigma$$

- Register updates:

$$r_1 \leftarrow \begin{cases} d & \text{if at } q_0 \text{ and } r_1 = 0 \\ r_1 - 1 & \text{if at } q_0 \text{ and } r_1 > 0 \\ r_1 & \text{if at } q_k, k \geq 1 \end{cases}$$

